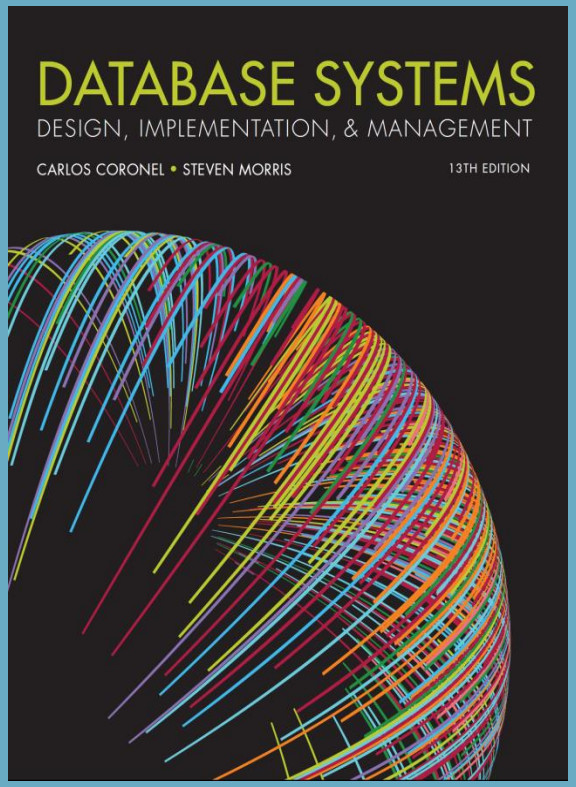


Chapter 8

Advanced SQL

Learning Objectives

- After completing this chapter, you will be able to:
  - Use SQL to create a table manually
  - Use SQL to create a copy of a table using a subquery
  - Manipulate the structure of existing tables to add, modify, and remove columns and constraints
  - Use SQL to do data manipulation (insert, update, and delete rows of data)
  - Use SQL to create database views, including updatable views

  - Next week……
  - Use Procedural Language SQL (PL/SQL) to create triggers, stored procedures, and PL/SQL functions

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Data Definition Commands (1 of 3)

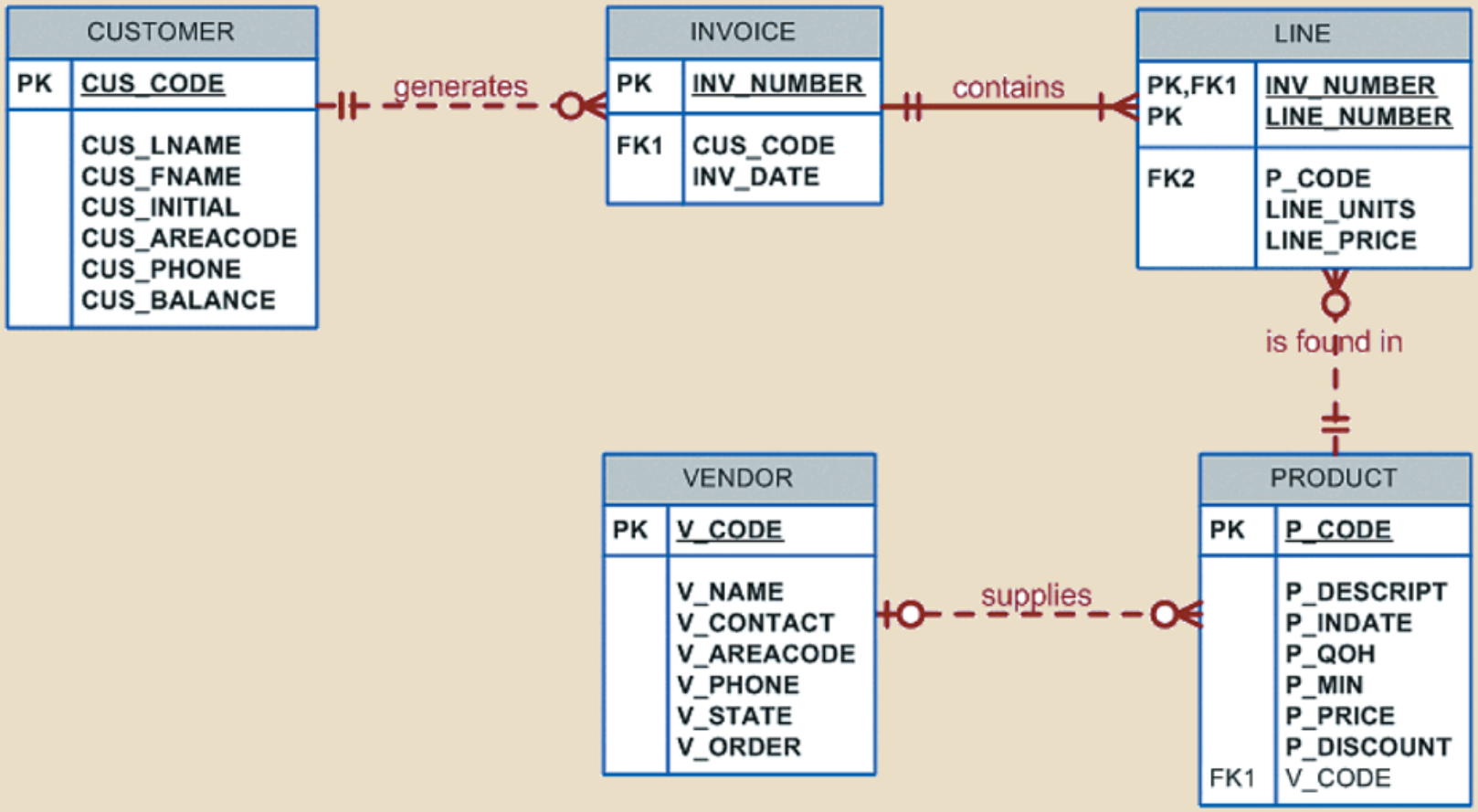
- Starting database model
  - Refer to Figure 8.1
- Creating the database
  - Before a new RDBMS can be used, the database structure and the tables that will hold the end-user data must be created
- The database schema
  - Logical group of database objects—such as tables and indexes—that are related to each other
- Data types
  - Character, numeric, and date

CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Data Definition Commands (2 of 3)

FIGURE 8.1 DATABASE MODEL



© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Data Definition Commands (3 of 3)

TABLE 8.1: Some Common SQL Data Types		
Data Type	**Format**	**Comments**
Numeric	NUMBER(L,D) or NUMERIC(L,D)	The declaration NUMBER(7,2) or NUMERIC(7,2) indicates that numbers will be stored with two decimal places and may be up to seven digits long, including the sign and the decimal place (for example, 12.32 or −134.99).
	INTEGER	May be abbreviated as INT. Integers are (whole) counting numbers, so they cannot be used if you want to store numbers that require decimal places.
	SMALLINT	Like INTEGER but limited to integer values up to six digits. If your integer values are relatively small, use SMALLINT instead of INT.
	DECIMAL(L,D)	Like the NUMBER specification, but the storage length is a minimum specification. That is, greater lengths are acceptable, but smaller ones are not. DECIMAL(9,2), DECIMAL(9), and DECIMAL are all acceptable.
Character	CHAR(L)	Fixed-length character data for up to 255 characters. If you store strings that are not as long as the CHAR parameter value, the remaining spaces are left unused. Therefore, if you specify CHAR(25), strings such as Smith and Katzenjammer are each stored as 25 characters. However, a U.S. area code is always three digits long, so CHAR(3) would be appropriate if you wanted to store such codes.
	VARCHAR(L) or VARCHAR2(L)	Variable-length character data. T he designation VARCHAR2(25) or VARCHAR(25) will let you store characters up to 25 characters long. However, unlike CHAR, VARCHAR will not leave unused spaces. Oracle automatically converts VARCHAR to VARCHAR2.
Date	DATE	Stores dates in the Julian date format.

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Creating Table Structures (1 of 2)

- CREATE TABLE command

```
CREATE TABLE tablename (
column1          data type        [constraint] [,
column2          data type        [constraint] ] [,
PRIMARY KEY      (column1         [, column2]) ] [,
FOREIGN KEY      (column1         [, column2]) REFERENCES tablename] [,
CONSTRAINT       constraint ] );
```

- SQL constraints
  - FOREIGN KEY
  - NOT NULL
  - UNIQUE
  - DEFAULT
  - CHECK

CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Creating Table Structures (2 of 2)

- Create a table with a SELECT statement
  - Rapidly creates a new table based on selected columns and rows of an existing table using a subquery
  - Automatically copies all of the data rows returned

```
-- COPYING PARTS OF TABLES
-- Create a PART table with only using the code, description and
price columns of PRODUCT
CREATE TABLE PART AS
SELECT P_CODE AS PART_CODE,
       P_DESCRIPT AS PART_DESCRIPT,
       P_PRICE AS PART_PRICE,
       V_CODE
FROM   PRODUCT;

SELECT * FROM PART;
```

	PART_CODE	PART_DESCRIPT	PART_PRICE	V_CODE
1	11QER/31	Power painter, 15 psi., 3-nozzle	109.99	25595
2	13-Q2/P2	7.25-in. pwr. saw blade	14.99	21344
3	14-Q1/L3	9.00-in. pwr. saw blade	17.49	21344
4	1546-QQ2	Hrd. cloth, 1/4-in., 2x50	39.95	23119
5	1558-QW1	Hrd. cloth, 1/2-in., 3x50	43.99	23119
6	2232/QTY	BeeDee jigsaw, 12-in. blade	109.92	24288
7	2232/QWE	BeeDee jigsaw, 8-in. blade	99.87	24288
8	2238/QPD	BeeDee cordless drill, 1/2-in.	38.95	25595
9	23109-HB	Claw hammer	9.95	21225
10	23114-AA	Sledge hammer, 12 lb.	14.4	(null)
11	54778-2T	Rat-tail file, 1/8-in. fine	4.99	21344
12	89-WRE-Q	Hicut chain saw, 16 in.	256.99	24288
13	PVC23DRT	PVC pipe, 3.5-in., 8-ft	5.87	(null)

CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Altering Table Structures (1 of 3)

- All changes in the table structure are made by using the ALTER TABLE command followed by a keyword that produces the specific change you want to make
  - ADD, MODIFY, and DROP
- Changing a column's data type
  - ALTER
- Changing a column's data characteristics
  - If the column to be changed already contains data, you can make changes in the column's characteristics if those changes do not alter the data type
- Adding a column
  - You can alter an existing table by adding one or more columns
  - Be careful not to include the NOT NULL clause for the new column

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Altering Table Structures (2 of 3)

- Adding primary key, foreign key, and check constraints
  - Primary key syntax:

```
ALTER TABLE   PART
ADD           PRIMARY KEY (PART_CODE);
```

  - Foreign key syntax:

```
ALTER TABLE   PART
ADD           FOREIGN KEY (V_CODE) REFERENCES  VENDOR;
```

  - Check constraint syntax:

```
ALTER TABLE   PART
ADD           CHECK (PART_PRICE >= 0);
```

  - Adding many constraints:

```
ALTER TABLE   PART
ADD           PRIMARY KEY(PART_CODE)
ADD           FOREIGN KEY (V_CODE) REFERENCES VENDOR;
```

CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Altering Table Structures (3 of 3)

- Dropping a column
  - Syntax:

```
ALTER TABLE VENDOR
DROP COLUMN V_ORDER;
```

- Deleting a table from the database
  - Syntax:

```
DROP TABLE PART;
```

CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

INDEXES (1 of 5)

- Indexes are used to improve the efficiency of searches and to avoid duplicate column values.

CREATE [UNIQUE] INDEX *indexname* ON *tablename*(*column1* [, *column2*])

```
CREATE INDEX P_INDATEX ON PRODUCT(P_INDATE);
```

- SQL does not let you write over an existing index without warning you first, thus preserving the index structure within the data dictionary.

To delete an index, use the DROP INDEX command:

DROP INDEX *indexname*

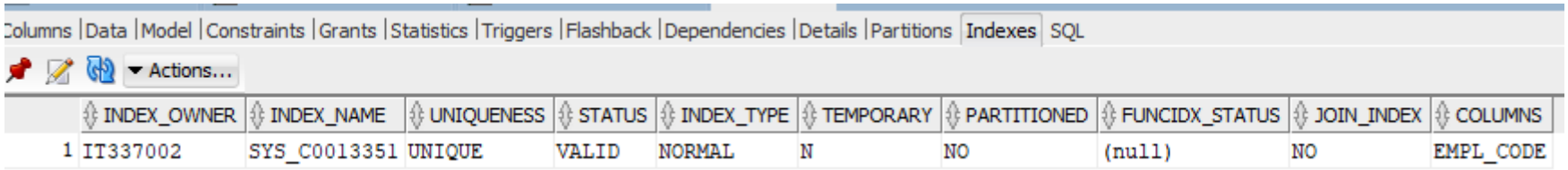
CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

INDEXES (2 of 5)

Using the UNIQUE index qualifier, you can even create an index that prevents you from using a value that has been used before. Such a feature is especially useful when the index attribute is a candidate key whose values must not be duplicated:

```
CREATE TABLE EMPL (
EMPL_CODE NUMBER(10) PRIMARY KEY,
EMPL_DRVNG_LSC VARCHAR2(20),
EMPL_TITLE VARCHAR2(4),
EMPL_LNAME VARCHAR2(15) NOT NULL,
EMPL_FNAME VARCHAR2(15) NOT NULL,
EMPL_INITIAL VARCHAR2(1),
EMPL_DOB DATE,
EMPL_JOINING_DATE DATE DEFAULT SYSDATE,
EMPL_SALARY NUMBER(10)
);
```



	INDEX_OWNER	INDEX_NAME	UNIQUENESS	STATUS	INDEX_TYPE	TEMPORARY	PARTITIONED	FUNCIDX_STATUS	JOIN_INDEX	COLUMNS
1	IT337002	SYS_C0013351	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_CODE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

INDEXES (3 of 5)

```
CREATE UNIQUE INDEX EMPL_DRVNG_LSC_X ON EMPL(EMPL_DRVNG_LSC);
```

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	INDEX_OWNER	INDEX_NAME	UNIQUENESS	STATUS	INDEX_TYPE	TEMPORARY	PARTITIONED	FUNCIDX_STATUS	JOIN_INDEX	COLUMNS
1	IT337002	SYS_C0013351	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_CODE
2	IT337002	INDX_EMPL_DRVNG_LSC	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_DRVNG

```
ALTER INDEX SYS_C0013351
RENAME TO INDX_EMPL_CODE;
```

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	INDEX_OWNER	INDEX_NAME	UNIQUENESS	STATUS	INDEX_TYPE	TEMPORARY	PARTITIONED	FUNCIDX_STATUS	JOIN_INDEX	COLUMNS
1	IT337002	INDX_EMPL_CODE	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_CODE
2	IT337002	INDX_EMPL_DRVNG_LSC	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_DRVNG

INDEXES (4 of 5)

```
CREATE INDEX INDX_EMPL_LNAME
ON EMPL(EMPL_LNAME);
```

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	INDEX_OWNER	INDEX_NAME	UNIQUENESS	STATUS	INDEX_TYPE	TEMPORARY	PARTITIONED	FUNCIDX_STATUS	JOIN_INDEX	COLUMNS
1	IT337002	INDX_EMPL_CODE	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_CODE
2	IT337002	INDX_EMPL_LNAME	NONUNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_LNAME
3	IT337002	INDX_EMPL_DRVNG_LSC	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_DRVNG_

```
CREATE INDEX INDX_EMPL_LNAME_SAL
ON EMPL(EMPL_LNAME, EMPL_SALARY);
```

INDEX_NAME	UNIQUENESS	STATUS	INDEX_TYPE	TEMPORARY	PARTITIONED	FUNCIDX_STATUS	JOIN_INDEX	COLUMNS
INDX_EMPL_CODE	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_CODE
INDX_EMPL_LNAME	NONUNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_LNAME
INDX_EMPL_DRVNG_LSC	UNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_DRVNG_LSC
INDX_EMPL_LNAME_SAL	NONUNIQUE	VALID	NORMAL	N	NO	(null)	NO	EMPL_LNAME, EMPL_SALARY

```
DROP INDEX EMPL_DRVNG_LSC_X;
```

INDEXES – (5 of 5) examples

```
-- Create index on P_INDATE
CREATE INDEX P_INDATEX ON PRODUCT(P_INDATE);
-- Create composite index on V_CODE and P_CODE
CREATE INDEX VENPRODX ON PRODUCT(V_CODE,P_CODE);
---Create index on P_PRICE descendent order
CREATE INDEX PROD_PRICEX ON PRODUCT(P_PRICE DESC);
-- Create composite index on V_CODE and P_CODE
CREATE INDEX C_INDATEX ON CUSTOMER(CUS_LNAME)
```

```
DROP INDEX C_INDATEX;
DROP INDEX  P_INDATEX ;
DROP INDEX VENPRODX;
DROP INDEX PROD_PRICEX;
```

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Data Manipulation Commands (1 of 5)

- Adding table rows
  - INSERT command syntax:

    INSERT INTO *tablename* VALUES (*value1, value2, …, valuen*)
  - Inserting rows with null attributes: use NULL entry
  - Inserting rows with optional attributes: indicate attributes that have required values
- Inserting table rows with a SELECT subquery
  - Add multiple rows to a table, using another table as the source, at the same time
  - SELECT syntax:

    INSERT INTO *target_tablename*[(*target_columnlist*)]
    SELECT *source_columnlist*
    FROM *source_tablename*;
- Saving table changes
  - COMMIT command syntax:

    COMMIT [WORK]

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Data Manipulation Commands (2 of 5)

```
CREATE TABLE V (
V_CODE                INTEGER PRIMARY KEY,
V_NAME                VARCHAR(35) NOT NULL,
V_CONTACT     VARCHAR(15) NOT NULL,
V_AREACODE    CHAR(3) NOT NULL,
V_PHONE       CHAR(8) NOT NULL,
V_STATE       CHAR(2) NOT NULL,
V_ORDER       CHAR(1) NOT NULL);

/* VENDOR rows                                                    */

INSERT INTO V (SELECT * FROM VENDOR);

SELECT * FROM V
```


© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Data Manipulation Commands (3 of 5)

- Updating table rows
  - UPDATE command is used to modify data in a table
  - UPDATE syntax:

UPDATE	*tablename*
SET	*columnname* = *expression* [, *columnname* = *expression*]
[WHERE	*conditionlist*];

```
UPDATE vendor SET v_phone = '222_3333' WHERE v_code = 21225;
```

CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Data Manipulation Commands (4 of 5)

- Deleting table rows
  - DELETE statement syntax:

DELETE FROM *tablename*
[WHERE *conditionlist*];

```
DELETE
FROM part
WHERE v_code = 21225;
```

IMPORTANT - For both update and delete, if the where condition is omitted, all records in the table will be updated or removed.

If a foreign key references a value being deleted, the DBMS will generate an error. The foreign key constraint must be disabled before the record can be deleted.

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Data Manipulation Commands (5 of 5)

- Restoring table contents
  - Undoes the changes since last COMMIT command
  - ROLLBACK command is used restore the database to its previous condition

ROLLBACK;

```
--COMMIT AND ROLLBACK
INSERT INTO VENDOR VALUES(11111,'ABCDEFGHIJK'
,'LMNOPQRST','222','222-2222','AA','Y');
SELECT * FROM VENDOR WHERE V_CODE = '11111';
COMMIT;
DELETE FROM VENDOR WHERE V_CODE = '11111';
SELECT * FROM VENDOR;
ROLLBACK;
SELECT * FROM VENDOR;
```

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Virtual Tables: Creating a View

- View: virtual table based on a SELECT query
  - Base tables: tables on which the view is based
- CREATE VIEW statement: data definition command that stores the subquery specification in the data dictionary
  - CREATE VIEW command syntax:

CREATE VIEW *viewname* AS SELECT *query*

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Virtual Tables: Creating a View

```
CREATE VIEW V_VENDOR AS
SELECT V_CODE VendorCode, V_NAME VendorName, V_PHONE VendorPhone
FROM VENDOR;

DESC V_VENDOR;

SELECT * FROM USER_VIEWS;

SELECT * FROM V_VENDOR;
```

	VENDORCODE	VENDORNAME	VENDORPHONE
1	21225	Bryson, Inc.	223-3234
2	21226	SuperLoo, Inc.	215-8995
3	21231	Drills Supply	228-3245
4	21344	Gomez Bros.	889-2546
5	22567	Dome Supply	678-1419
6	23119	Randsets Ltd.	678-3998
7	24004	Brackman Bros.	228-1410
8	24288	ORDVA, Inc.	898-1234
9	25443	Big Save, Inc.	227-0093
10	25501	Damal Supplies	890-3529
11	25595	Rubicon Systems	456-0092

CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.



Updatable Views

- Used to update attributes
  - Batch update routine: pools multiple transactions into a single batch to update a master table field in a single operation
- Updatable view restrictions
  - GROUP BY expressions or aggregate functions cannot be used
  - Set operators cannot be used
  - Most restrictions are based on the use of JOINs or group operators in views

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Updatable Views

Two tables PRODMASTER (master product) and PRODSALES (monthly sales) have a 1: 1 relationship

PROD_ID	PROD_DESC	PROD_QOH
A123	SCREWS	67
BX34	NUTS	37
C583	BOLTS	50

PROD_ID	PS_QTY
A123	7
BX34	3

```
CREATE OR REPLACE VIEW PSVUPD AS (
SELECT PRODMASTER.PROD_ID AS PRODUCT_ID, PRODMASTER.PROD_QOH AS
PRODUCT_QOH, PRODSALES.PS_QTY AS SALES_QTY
FROM PRODMASTER, PRODSALES
WHERE PRODMASTER.PROD_ID = PRODSALES.PROD_ID);
```

```
SELECT * FROM PSVUPD;
```

PRODUCT_ID	PRODUCT_QOH	SALES_QTY
A123	67	7
BX34	37	3



Create Updateable View

PRODMASTER is updated by subtracting PS_QTY , from PRODSALES.

```
UPDATE PSVUPD

SET PRODUCT_QOH = PRODUCT_QOH - SALES_QTY;
```

PRODMASTER table has been updated

```
SELECT * FROM PRODMASTER;
```

PROD_ID	PROD_DESC	PROD_QOH
A123	SCREWS	60
BX34	NUTS	34
C583	BOLTS	50



VIEWS

EXAMPLE – SUMMARY OF TOTAL PRODUCT COST AND QUANTITY ON HAND STATISTICS GROUPED BY VENDOR

```
CREATE OR REPLACE VIEW VW_PROD_STATS AS

SELECT V_CODE, SUM(P_QOH * P_PRICE) AS TOTCOST, MAX(P_QOH)
AS MAXQTY, MIN(P_QOH) AS MINQTY, AVG(P_QOH) AS AVGQTY

FROM PRODUCT

GROUP BY V_CODE;


SELECT * FROM VW_PROD_STATS;
```

VIEWS

```
CREATE OR REPLACE VIEW VW_AREACODE AS
        SELECT V_CODE, V_NAME, V_CONTACT, V_AREACODE AS VW1, V_PHONE, V_STATE, V_ORDER
                FROM VENDOR
                WHERE V_AREACODE = '615';

SELECT * FROM VW_AREACODE;
INSERT INTO VW_AREACODE
                (V_CODE, V_NAME, V_CONTACT, VW1,  V_PHONE, V_STATE, V_ORDER)
                VALUES (22222, 'AASSDDFFGG', 'XXXXX', '615', '123-4321', 'NZ', 'Y');

SELECT * FROM VW_AREACODE;

DROP VIEW VW_AREACODE;
```

CENGAGE

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Sequences (1 of 2)

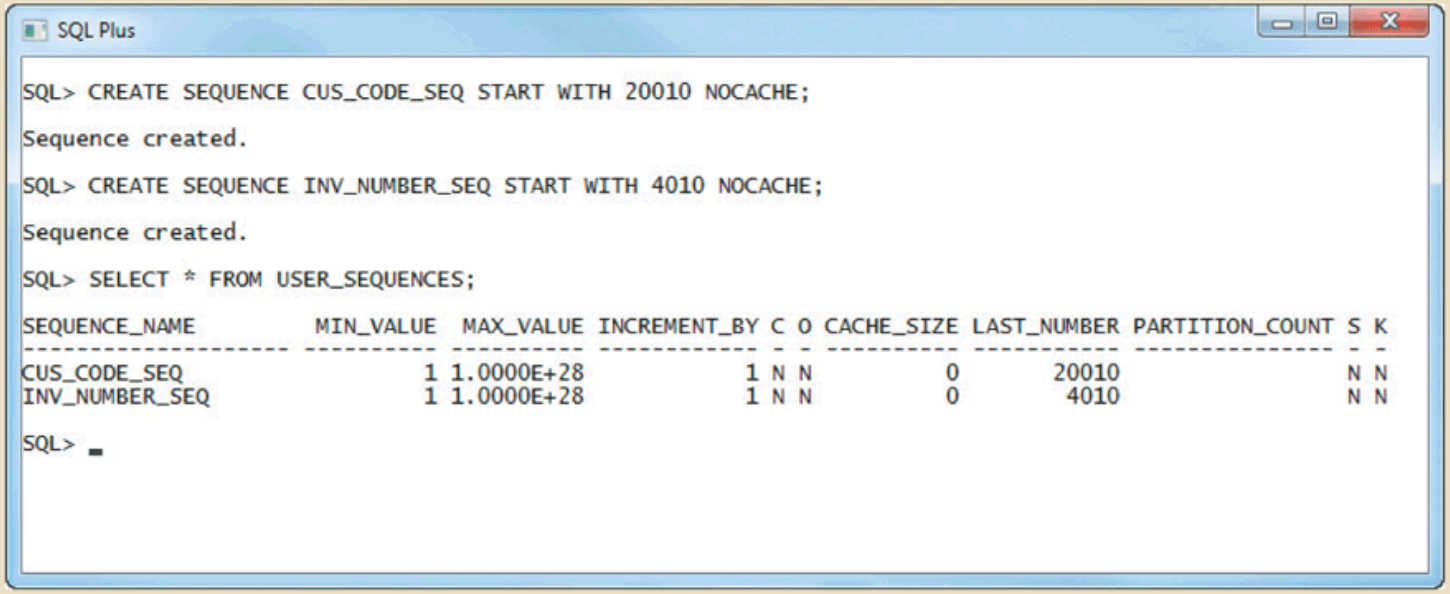
- Many similarities in the use of sequences across these DBMSs
  - Independent object in the database
  - Have a name and can be used anywhere a value expected
  - Not tied to a table or column
  - Generate a numeric value that can be assigned to any column in any table
  - Table attribute with an assigned value can be edited and modified

- Sequences are widely used to generate primary keys, such as account numbers, and for internal control purposes.
  - When a sequence is used to generate account numbers, most organizations will try to provide some diversity among the number so that a customer does not accidentally access another account.
  - NEXTVAL – will give the next value of the sequence
  - CURRVAL – will identify the last value generated.

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Sequences (2 of 2)

FIGURE 8.10 ORACLE SEQUENCE



© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.



Oracle Sequences

```
CREATE SEQUENCE CUS_CODE_SEQ START WITH 20010 NOCACHE;

INSERT INTO CUSTOMER VALUES (CUS_CODE_SEQ.NEXTVAL,
'Connery', 'Sean', NULL, '615', '898-2007', 0.00);

SELECT * FROM USER_SEQUENCES;

SELECT CUS_CODE_SEQ.CURRVAL from DUAL;

DROP SEQUENCE CUS_CODE_SEQ;
```



DUAL Table

- The DUAL table is a dummy table created by default in Oracle along with the data dictionary.
- The table consists of one column and one row and stores no data.
- Since a SELECT statement must reference a table in the FROM clause, the DUAL table is frequently used to return values from a function when the value is not already stored in a database table. It helps user-written programs to check and then guarantee a known result.

DUAL Table

- DUAL table has one row – though its name is misleading. It originally had two rows in it, but subsequently the later versions of Oracle changed to have one row.

```
SQL> desc dual
 Name                      Null?    Type
 ------------------------- -------- ----------------------------
 DUMMY                               VARCHAR2(1)
```

```
select * from dual;


select months_between('20-april-2010', '01-jan-1984')
       from dual;


select sysdate from dual;


select user from dual;
```



Virtual Columns

- Virtual columns appear to be normal table columns, but their values are derived rather than being stored on disc.

column_name [datatype] [GENERATED ALWAYS] AS (expression) [VIRTUAL]

However, note that virtual columns can be used only for a deterministic user-defined function. i.e., A function which returns same value irrespective of whenever you call it.



Virtual Columns

```
CREATE TABLE class
   (
     class_id        NUMBER(6,3) NOT NULL,
     class_Title     VARCHAR(20) NOT NULL,
     class_Location VARCHAR2(10) NOT NULL,
     campus          VARCHAR2(4) generated always AS (
     CASE
       WHEN (SUBSTR(class_location,1,1) = 'A')
  THEN 'ALBN'
  ELSE 'PNTH'
END)
Virtual)
```

Now try this

```
insert into class (class_id, class_title, class_location)
values (158.337, 'Database Development', 'Albany');
commit;
select * from class;
```



Format

Try formatting PRODUCT table as follows:

```
COLUMN P_CODE FORMAT A8
COLUMN P_DESCRIPT FORMAT A14 TRUNCATED
COLUMN P_QOH FORMAT 9999
COLUMN P_MIN FORMAT 9999
COLUMN P_PRICE FORMAT 999.99
COLUMN P_DISCOUNT FORMAT 0.99
COLUMN V_CODE FORMAT 99999
COLUMN P_MIN_ORDER FORMAT 9999
COLUMN P_REORDER FORMAT 9
SELECT * FROM PRODUCT
```

Retrieving Data From a Single Database Table

- Where
- Group By
- Having
- Order By

```
SELECT   [DISTINCT | UNIQUE] (*, columnname [ AS alias], …)
         FROM       tablename
         [WHERE     condition]
         [GROUP BY  group_by_expression]
         [HAVING    group_condition]
         [ORDER BY  columnname];
```

group_by_expression forms groups of rows with the same value

group_condition filters the groups subject to some condition

columnname(s) specifies the order of the output

Retrieve Data – WHERE clause

- IN comparison operator
  - Match data values that are members of a set of search values

- LIKE operator
  - Use to match part of character string
    - WHERE *columnname* LIKE '*string*'
    - The LIKE operator is different from the other basic comparison operators in that it is used to identify character patterns. The % and _ wildcard characters are used to create a pattern.

```
SELECT V_NAME, V_CONTACT, V_AREACODE, V_PHONE
FROM   VENDOR
WHERE  V_CONTACT LIKE 'Smith%'
```

Comparison Operators

Mathematical Comparison Operators	
=	Equality or "equal to"—for example, cost = 55.95
>	Greater than—for example, cost > 20
<	Less than—for example, cost < 20
<>, !=, or ^=	Not equal to—for example, cost <> 55.95 or cost != 55.95 or cost ^= 55.95
<=	Less than or equal to—for example, cost <= 20
>=	Greater than or equal to—for example, cost >= 20



WHERE Subqueries

- Uses inner SELECT subquery on the right side of a WHERE comparison expression
- Value generated by the subquery must be of a comparable data type
- Can be used in combination with joins

Eg., to return a list of vendors who have not supplied products

```
SELECT V_CODE, V_NAME FROM VENDOR
WHERE V_CODE NOT IN (SELECT V_CODE FROM PRODUCT);
```



IN Subqueries

- IN subqueries
  - Used to compare a single attribute to a list of values

    Eg., to return a list of vendors who have supplied products

```
SELECT V_CODE, V_NAME FROM VENDOR
WHERE V_CODE NOT IN (SELECT V_CODE FROM PRODUCT);
```



HAVING Subqueries

- HAVING subqueries
  - HAVING clause restricts the output of a GROUP BY query by applying conditional criteria to the grouped rows
  - Eg., to list all products with total quantity sold greater than average quantity sold

```
SELECT P_CODE, SUM(LINE_UNITS) FROM LINE
GROUP BY P_CODE
HAVING SUM(LINE_UNITS) > (SELECT AVG(LINE_UNITS) FROM
LINE);
```



Multirow Subquery Operators: ANY and ALL

- ALL operator
  - Allows comparison of a single value with a list of values returned by the first subquery
    - Uses a comparison operator other than equals

```
SELECT P_CODE, P_QOH * P_PRICE
FROM PRODUCT
WHERE P_QOH * P_PRICE > ALL(SELECT P_QOH * P_PRICE
                            FROM PRODUCT
                            WHERE V_CODE IN
                  (SELECT V_CODE FROM VENDOR WHERE V_STATE = 'FL'));
```



Multirow Subquery Operators: ANY and ALL

- ANY operator
  - Allows comparison of a single value to a list of values and selects only the rows for which the value is greater than or less than any value in the list

```
SELECT P_CODE,  P_QOH * P_PRICE
FROM PRODUCT
WHERE P_QOH     * P_PRICE > ANY  (SELECT P_QOH * P_PRICE
                                   FROM PRODUCT
                                  WHERE V_CODE IN
                (SELECT V_CODE FROM VENDOR WHERE V_STATE = 'FL'));
```



SELF-JOIN

- A Self-Join joins a table to itself by pretending there are different tables involved. This is accomplished by using table aliases, i.e. one table has one alias, and the same table also has another alias.
- A Self-Join is needed to report on recursive data
- e.g. COURSE (COURSE_NO, COURSE_DESC, COURSE_PREREQ)
- --- to find the pre-requisite course
- --- 158.337 has pre-requisite 158.2XX

- e.g. EMPLOYEE (EMP_NO, EMP_NAME, EMP_MNGR_NUM)
- --- to find the manager of an employee
- --- Sam (EMP_NUM = 21) reports to manager Linda (EMP_NUM = 18)
-

Recursive Joins

- Table EMP
- Each Employee has a manager. The manager is hired on a certain date.
- You have to compare all employees (who are not managers) with their Manager's hire date.

	COLUMN_NAME	DATA_TYPE
1	EMP_NUM	NUMBER
2	EMP_TITLE	CHAR(10 BYTE)
3	EMP_LNAME	VARCHAR2(15 BYTE)
4	EMP_FNAME	VARCHAR2(15 BYTE)
5	EMP_INITIAL	CHAR(1 BYTE)
6	EMP_DOB	DATE
7	EMP_HIRE_DATE	DATE
8	EMP_AREACODE	CHAR(3 BYTE)
9	EMP_PHONE	CHAR(8 BYTE)
10	EMP_MGR	NUMBER

Recursive Query - Display the names of all employees hired prior to when the manager of their branch was hired.

Identify sub-query

	COLUMN_NAME	DATA_TYPE
1	EMP_NUM	NUMBER
2	EMP_TITLE	CHAR(10 BYTE)
3	EMP_LNAME	VARCHAR2(15 BYTE)
4	EMP_FNAME	VARCHAR2(15 BYTE)
5	EMP_INITIAL	CHAR(1 BYTE)
6	EMP_DOB	DATE
7	EMP_HIRE_DATE	DATE
8	EMP_AREACODE	CHAR(3 BYTE)
9	EMP_PHONE	CHAR(8 BYTE)
10	EMP_MGR	NUMBER

EMP A

	COLUMN_NAME	DATA_TYPE
1	EMP_NUM	NUMBER
2	EMP_TITLE	CHAR(10 BYTE)
3	EMP_LNAME	VARCHAR2(15 BYTE)
4	EMP_FNAME	VARCHAR2(15 BYTE)
5	EMP_INITIAL	CHAR(1 BYTE)
6	EMP_DOB	DATE
7	EMP_HIRE_DATE	DATE
8	EMP_AREACODE	CHAR(3 BYTE)
9	EMP_PHONE	CHAR(8 BYTE)
10	EMP_MGR	NUMBER

EMP B

Call Table "A" – to find employee details. These employees are not managers

```
SELECT A.EMP_LNAME ||','|| A.EMP_FNAME "Employee Full Name" , A.EMP_HIRE_DATE "Date Employee Hired"
FROM EMP A
```

Call Table "B" – to find Manager's hire date `SELECT B.EMP_HIRE_DATE FROM EMP B WHERE B.EMP_NUM = A.EMP_MGR`

Place this with the employee details

```
SELECT A.EMP_LNAME ||','|| A.EMP_FNAME "Employee Full Name" , A.EMP_HIRE_DATE "Date Employee Hired",
(SELECT B.EMP_HIRE_DATE FROM EMP B WHERE B.EMP_NUM = A.EMP_MGR) "Date Manager Hired"
FROM EMP A
```

Next compare the hire date in WHERE clause

- Use the sub-query in the column list to identify Date the Manager was hired
- Again use sub-query in WHERE clause to make comparisons with the date

```
SELECT A.EMP_LNAME ||','|| A.EMP_FNAME "Employee Full Name" ,
A.EMP_HIRE_DATE "Date Employee Hired",
      (SELECT B.EMP_HIRE_DATE FROM EMP B WHERE B.EMP_NUM =
      A.EMP_MGR) "Date Manager Hired"
       FROM EMP A
WHERE A.EMP_MGR IS NOT NULL
AND A.EMP_HIRE_DATE < (SELECT B.EMP_HIRE_DATE FROM EMP B
                       WHERE  B.EMP_MGR IS NULL
                       AND B.EMP_NUM = A.EMP_MGR);   )
```

	Employee Full Name	Date Employee Hired	Date Manager Hired
1	Lange,John	20/10/94	08/11/98
2	Smith,Jeanine	05/01/89	08/11/98
3	Diante,Jorge	02/07/94	08/11/98
4	Smith,George	14/04/89	18/11/92
5	Genkazi,Leighla	01/12/90	18/11/92



Summary

- The ANSI standard data types are supported by all RDBMS vendors in different ways
- The basic data definition commands allow you to create tables and indexes
- Data manipulation commands allow you to add, modify, and delete rows from tables
- Views can be created to expose subsets of data to end users primarily for security and privacy reasons
- In Oracle and SQL Server, sequences may be used to generate values to be assigned to a record

© 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.